SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1105

Course Outcome Covered

Course Code & Name	**CSA1105 – Object Oriented Analysis and Design**
Assignment Title	**Object-Oriented Analysis and UML-Based Design of a Smart Warehouse Inventory and Order Fulfilment System (SWIFT)**
Course Outcome(s) – CO	**CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.** **CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.** **CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.**
Bloom's Taxonomy Level	**BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create**

SDG Mapping (SDG 1–17)	**SDG 9 – Industry, Innovation and Infrastructure;** **SDG 12 – Responsible Consumption and Production**
Industry / Societal Relevance	**Warehouse automation, logistics and supply-chain digitalisation, Industry 4.0, and software-based inventory/fulfilment management**

Assignment Weightage: 100%

QUESTIONS: 1 **Total Marks: 100**

Annexure A

Assignment

Code of Conduct:

I _____RASHMI NAURENE (192521357)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____  **_____**

1. Problem Analysis and Requirement Summary

The warehouse currently tracks inventory, storage bins, and orders manually using registers and spreadsheets. As order volumes grow, this manual approach causes bin double-allocation, orders confirmed against out-of-stock items, missed dispatch SLAs, poor zone-utilisation visibility, uneven picker workload, and an inability to trace an order's status or analyse fulfilment performance. SWIFT is the proposed object-oriented software solution that manages inventory, storage allocation, order lifecycle, picking, and performance reporting to remove these manual-process failure points.

Problem Domain Analysis

Each of the seven failure points named in the case study traces back to the same root cause: the manual process has no single, authoritative record of state that every operation checks before acting. A storage bin can be double-allocated because two operators consult the same paper log without either seeing the other's pending update; an order can be confirmed against out-of-stock items because the person taking the order has no live view of what a picker is simultaneously removing from a bin. An object-oriented design addresses this directly by making each piece of state (stock level, bin occupancy, order status) the responsibility of exactly one class, so every part of the system that needs to check or change that state goes through the same object rather than through a separate, unsynchronised copy of the information.

This reframes the problem in object-oriented terms from the outset: the warehouse is not a set of spreadsheets to digitise, but a set of collaborating objects (items, bins, orders, picking sessions) that must enforce their own consistency rules — which is exactly what Sections 5, 13, and 15 of this report formalise.

Stakeholders

- **Operator (warehouse/inventory clerk) —** the primary internal user; responsible for keeping the item catalogue and storage-bin state accurate, and for producing the fulfilment report management uses to judge warehouse performance.
- **Customer —** the external party whose order must be confirmed only when it can actually be fulfilled, and who needs visibility into where their order stands.
- **Picker —** the internal user who physically executes fulfilment, and whose workload and exception reports are direct inputs to the reporting requirement (FR6).
- **Warehouse management (indirect stakeholder) —** does not interact with the system directly but is the consumer of the fulfilment report and the party ultimately accountable for the SLA-compliance and utilisation figures it contains.

Major System Responsibilities

Read together, the six functional requirements in the table below imply four system-level responsibilities that recur throughout the rest of this report: (1) maintain a single consistent record of what stock exists and where it is stored; (2) prevent any operation — an allocation, an order confirmation, a picker assignment — from committing to a resource it does not actually have available; (3) track every order through a well-defined lifecycle rather than an ad hoc status field; and (4) derive performance insight from the same operational records rather than a separately maintained reporting process. These four responsibilities are what the class structure in Section 5 is organised around.

Functional Requirements Summary

ID	Requirement	Summary
FR1	Item and Inventory Management	Store Item ID, name, category, unit price, reorder threshold, current stock; prevent duplicate IDs; flag reorder when stock falls below threshold.
FR2	Storage Zone / Bin Management	Store Bin ID, max capacity, current occupancy, stored item(s), availability; never allocate beyond available capacity.
FR3	Order Management	Create an order with Order ID, customer reference, line items, priority, status, timestamp; never confirm a line item exceeding available stock.
FR4	Picking and Fulfilment Session	Create a picking session with pick-list ID, assigned picker, items, status, start/completion time; move to Exception on damaged/missing items.
FR5	Order Status Tracking	Track an order through Placed → Allocated → Picking → Packed → Dispatched → Delivered, with alternate paths to Cancelled or Exception.
FR6	Reporting	Generate a report of total orders processed, SLA compliance %, most/least utilised zone, items due for reorder, and average picking time.

Objects Emerging from the Problem Domain (Preview)

Reading the requirement summary above against the stakeholders and responsibilities identified so far already surfaces the core objects this design is built around — Item, StorageBin, Order, OrderLineItem, PickingSession, Picker, and Customer — each corresponding to a noun the case study uses to describe something the system must remember and act on. Section 5 formalises this through a full noun/verb analysis; it is previewed here to show that the object identification is grounded in the problem statement itself rather than introduced later without justification.

2. Actor Identification

An actor is any external entity — a human role or another system — that interacts with SWIFT to achieve a goal but lies outside the system boundary. Applying this test to the problem statement (asking, for each candidate, "does this entity initiate or receive a use case, without being part of the system's own internal logic?") identifies exactly three actors.

Identified Actors

Actor	Role and Justification
Operator	Warehouse/inventory clerk who registers items, receives stock, allocates storage bins, tracks order status, and generates the fulfilment report. Identified directly from the case study's instruction that "the system should allow the operator to register items, receive stock, allocate storage bins..." — the operator is the actor with the widest range of use cases since most inventory-facing responsibilities fall to this role.
Customer	Places orders and tracks the status of their own orders. Identified from the order-placement and status-tracking requirements (FR3, FR5); the customer is external to the warehouse's own staff and only ever interacts with the order-facing part of the system, never with inventory or picking directly.
Picker	Executes assigned picking sessions and reports exceptions such as damaged or missing items. Identified from FR4's explicit mention of "assigned picker" and pick status; the picker's interaction is scoped entirely to the Pick Order use case and its exception path, distinguishing this role from the Operator's broader inventory responsibilities.

Actor Boundary Decisions

Two candidates were deliberately excluded from the actor list because they do not meet the external-interaction test above. A separate "Administrator" actor was considered but not introduced, since every administrative responsibility mentioned in the case study (registering items, managing bins, generating reports) already falls to the Operator; introducing a fourth actor purely to separate these functions was judged to add modelling overhead without a corresponding requirement to justify it. Similarly, the storage bin and the warehouse's physical layout are treated purely as system-internal state (represented by the StorageBin class in Section 5), not as an actor, since they never initiate a use case themselves — they are acted upon, not an active participant.

3. Use Case Descriptions

Formal descriptions are given below for the seven primary use cases shown in the Use Case Diagram (Section 4). Each include/extend relationship identified during use-case analysis is captured inside the relevant main or alternate flow.

UC-1: Register Item

Use Case	Register Item
Primary Actor(s)	Operator
Description	Allows the operator to add a new item to the inventory catalogue.
Preconditions	Operator is logged in; the item's ID does not already exist in the system.
Flows	**Main Flow:** 1. Operator enters item ID, name, category, unit price, and reorder threshold. 2. System checks that the item ID is not a duplicate. 3. System creates the item record with current stock initialised to zero. **Alternate / Exception Flow:** If the item ID already exists, the system rejects the request and asks the operator to use Receive Stock instead. **Postconditions:** A new Item record exists in the catalogue.

UC-2: Receive Stock

Use Case	Receive Stock
Primary Actor(s)	Operator
Description	Allows the operator to record newly arrived stock against an existing item.
Preconditions	The item already exists in the catalogue.
Flows	**Main Flow:** 1. Operator selects the item and enters the received quantity. 2. System increases Item.currentStock by the received quantity. 3. System checks whether the item is still below its reorder threshold. **Alternate / Exception Flow:** If the item does not exist, the operator is redirected to Register Item first. **Postconditions:** Item.currentStock is updated; the reorder flag is cleared if stock is now sufficient.

UC-3: Allocate Storage Bin

Use Case	Allocate Storage Bin
Primary Actor(s)	Operator
Description	Assigns received stock to a storage bin, subject to the bin's available capacity.
Preconditions	The item exists and has stock awaiting allocation; the target bin exists.
Flows	**Main Flow:** 1. Operator selects a bin for the item and quantity.

	2. System executes the included Check Bin Capacity sub-flow. 3. System updates the bin's current occupancy and links the item to the bin. **Alternate / Exception Flow:** <<include>> Check Bin Capacity: if the requested quantity would exceed the bin's available capacity, the allocation is rejected and the operator must choose a different bin. **Postconditions:** StorageBin.currentOccupancy reflects the newly allocated stock.

UC-4: Place Order

Use Case	Place Order
Primary Actor(s)	Customer
Description	Allows a customer to create an order containing one or more line items at a chosen priority.
Preconditions	The customer is registered; requested items exist in the catalogue.
Flows	**Main Flow:** 1. Customer selects items and quantities and chooses a priority (Standard/Express). 2. System executes the included Check Stock Availability sub-flow for every line item. 3. System creates the Order and its OrderLineItem records with status 'Placed'. **Alternate / Exception Flow:** <<include>> Check Stock Availability: if any line item's quantity exceeds available stock, the order is not confirmed and the customer is asked to adjust the affected line item. **Postconditions:** A new Order exists with status 'Placed' and is ready to progress to Allocated.

UC-5: Pick Order

Use Case	Pick Order
Primary Actor(s)	Picker
Description	Represents a picker executing an assigned picking session for a confirmed order.
Preconditions	The order has status 'Allocated' and a picking session has been created and assigned to this picker.
Flows	**Main Flow:** 1. Picker marks each item on the pick list as picked. 2. System updates PickingSession status through Assigned → In-Progress. 3. Picker marks the session complete once all items are picked; system sets Order status to 'Packed'.

	Alternate / Exception Flow: <<extend>> Report Exception: if an item is found damaged or missing, the picker reports it, and the session moves to the Exception state instead of completing normally. **Postconditions:** The picking session is Completed (or in Exception), and Order.status reflects the outcome.

UC-6: Track Order Status

Use Case	Track Order Status
Primary Actor(s)	Customer, Operator
Description	Allows the customer or operator to view an order's current position in its lifecycle.
Preconditions	The order exists in the system.
Flows	**Main Flow:** 1. Actor selects an order. 2. System retrieves the order's current status and history of transitions. 3. System displays the status to the actor. **Alternate / Exception Flow:** None. **Postconditions:** No state change; this is a read-only use case.

UC-7: Generate Fulfilment Report

Use Case	Generate Fulfilment Report
Primary Actor(s)	Operator
Description	Produces a performance report summarising fulfilment activity over a period.
Preconditions	At least one order has been processed within the reporting period.
Flows	**Main Flow:** 1. Operator selects a reporting period. 2. System aggregates total orders processed, SLA compliance percentage, zone utilisation, reorder list, and average picking time. 3. System displays/exports the report. **Alternate / Exception Flow:** If no orders exist for the period, the system returns an empty report rather than an error. **Postconditions:** A FulfilmentReport view is produced (as a derived/computed view, not a persisted entity — see Section 15).

4. Use Case Diagram

The diagram below models the three actors and seven primary use cases from Sections 2 and 3, together with two include relationships (mandatory sub-flows) and one extend relationship (the optional exception path during picking).

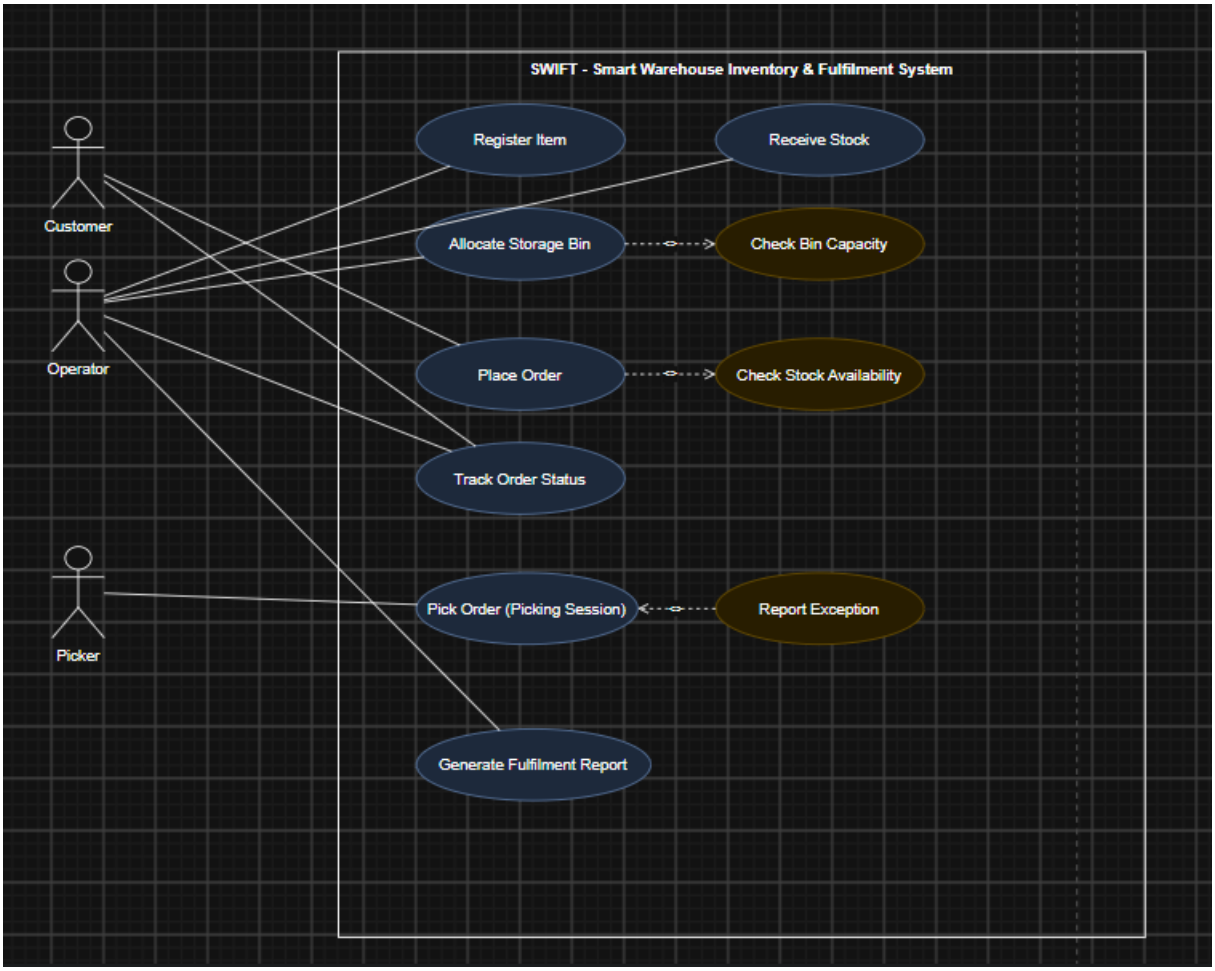

Figure 1. Use Case Diagram for SWIFT.

5. Object and Class Identification

Noun/Verb Analysis

Analysis	Result
Nouns → Candidate Classes	item, storage bin/zone, order, order line item, picking session, picker, customer, (fulfilment) report
Verbs → Candidate Operations	register, receive (stock), allocate, place (order), confirm, cancel, pick, track, generate, flag (exception)

Entity / Boundary / Control Classification

Object	Classification	Justification
Item, StorageBin, Order, OrderLineItem, PickingSession, Picker, Customer	Entity	Each holds persistent data that outlives a single interaction (stock levels, bin occupancy, order status).
OrderUI, PickingUI	Boundary	Represent the screen a Customer or Picker interacts with directly, isolating presentation from business logic.
OrderController, PickingController	Control	Coordinate the workflow between a boundary object and the entity objects it must update.

Classes, Attributes, and Operations

Class	Key Attributes	Key Operations
Customer	customerId, name	placeOrder()
Order (base)	orderId, priority, status, createdAt	confirmOrder(), cancelOrder()
StandardOrder / ExpressOrder	— (inherit Order)	getDispatchDeadline() — overridden: 24 hrs / 4 hrs
OrderLineItem	lineItemId, quantity	getSubtotal()
Item	itemId, name, unitPrice, reorderThreshold, currentStock	checkReorderNeeded(), updateStock()
StorageBin	binId, maxCapacity, currentOccupancy, status	checkAvailability(), allocate()
PickingSession	pickListId, status, startTime, completionTime	startPicking(), completePicking(), flagException()
Picker	pickerId, name, shift	getActiveSession()

Relationships and multiplicities (shown in Figure 2): Customer places Order (1 to 0..*); Order is composed of OrderLineItem (1 to 1..*); OrderLineItem references Item (0..* to 1); Item is stored in StorageBin (1 to 0..*); Order is fulfilled by PickingSession (1 to 0..1); PickingSession is assigned to Picker (0..* to 1); StandardOrder and ExpressOrder generalise from Order.

6. Class Diagram

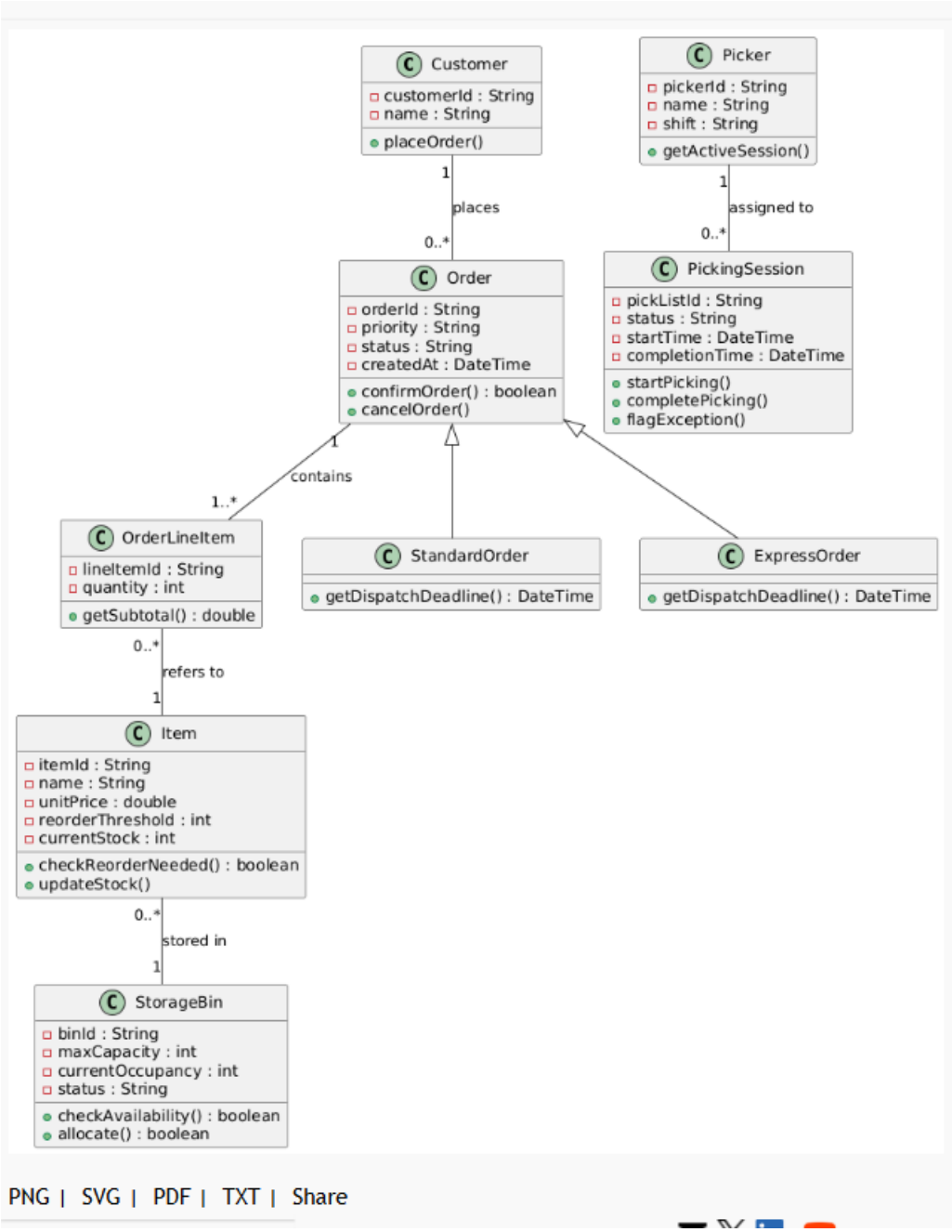

Figure 2. Class Diagram for SWIFT.

7. Sequence / Interaction Diagrams

Place Order

Customer submits an order through OrderUI, which hands off to OrderController. The controller checks stock on Item for each line item before creating the Order.

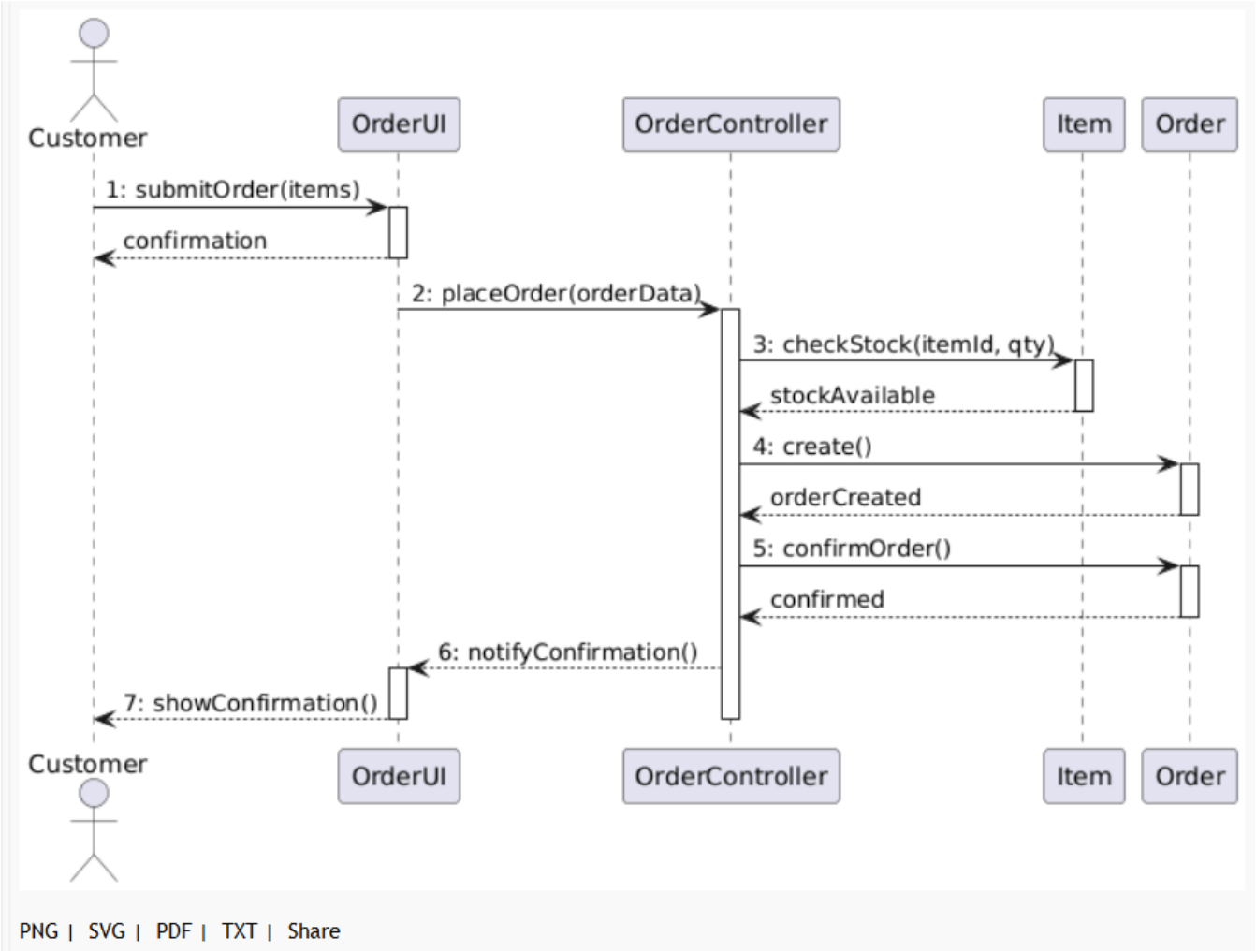

Figure 3. Sequence Diagram: Place Order.

Complete Picking Session

The Picker marks items as picked through PickingUI; once all items are picked, PickingController marks the session complete and updates the Order's status to 'Packed'.

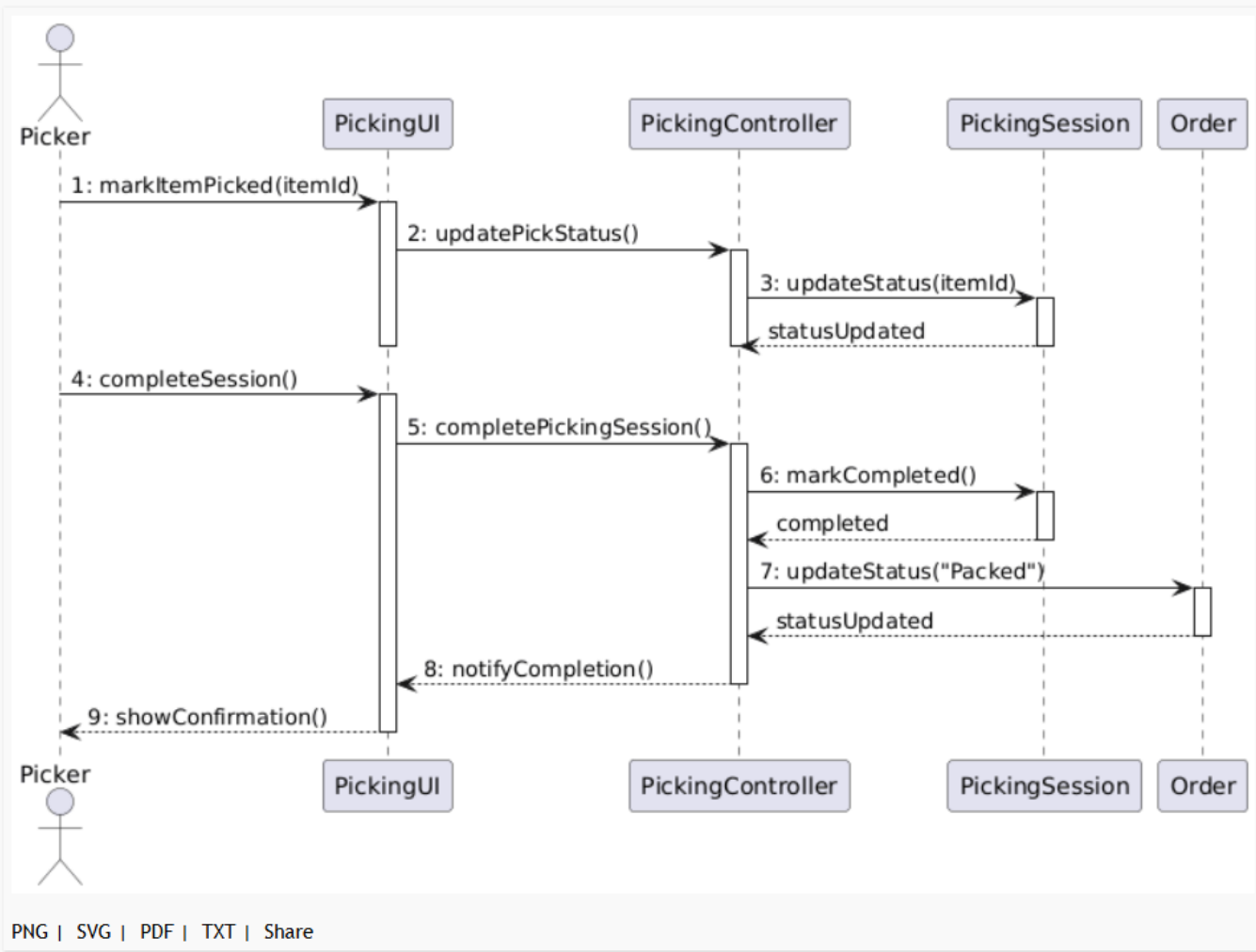

Figure 4. Sequence Diagram: Complete Picking Session.

8. Activity Diagram

Models the end-to-end order fulfilment workflow with two decision points: a stock-availability check immediately after order placement, and an item-condition check during picking that can loop back for a re-pick.

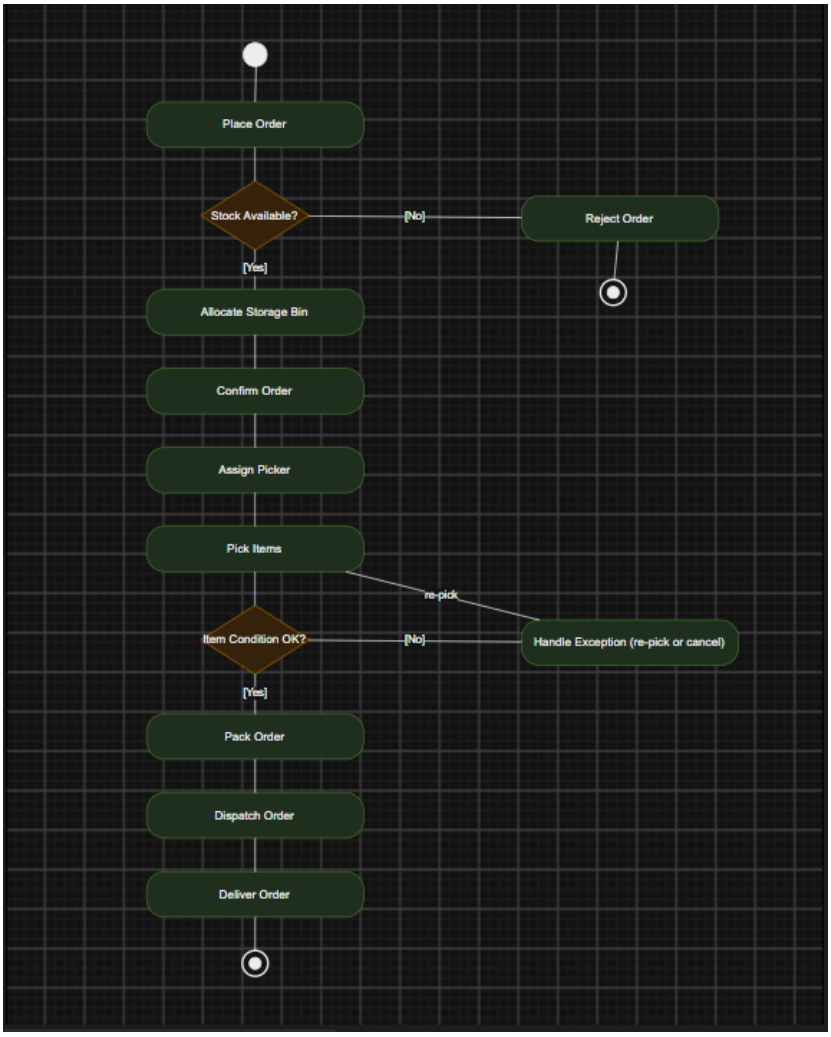

Figure 5. Activity Diagram: End-to-End Order Fulfilment.

9. State Transition Diagram

The Order object moves through Placed → Allocated → Picking → Packed → Dispatched → Delivered on the happy path, with alternate transitions to Cancelled and Exception exactly as required by FR5.

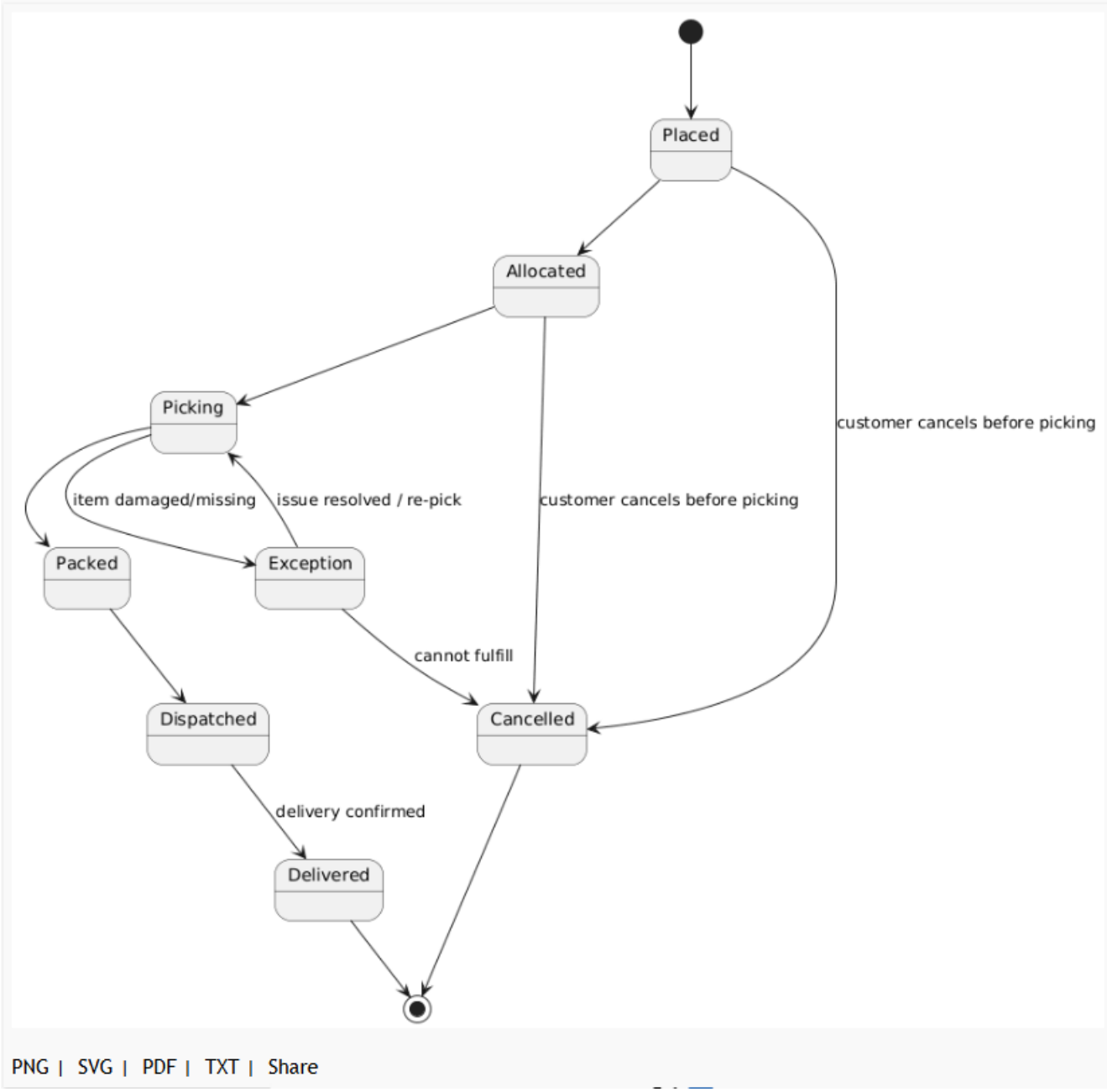

Figure 6. State Transition Diagram: Order Lifecycle.

10. Package Diagram

The system is grouped into five logical subsystems, with dependency arrows showing that Order Management depends on Inventory Management, Picking & Fulfilment depends on both, and Reporting depends on both Order Management and Picking & Fulfilment for its source data.

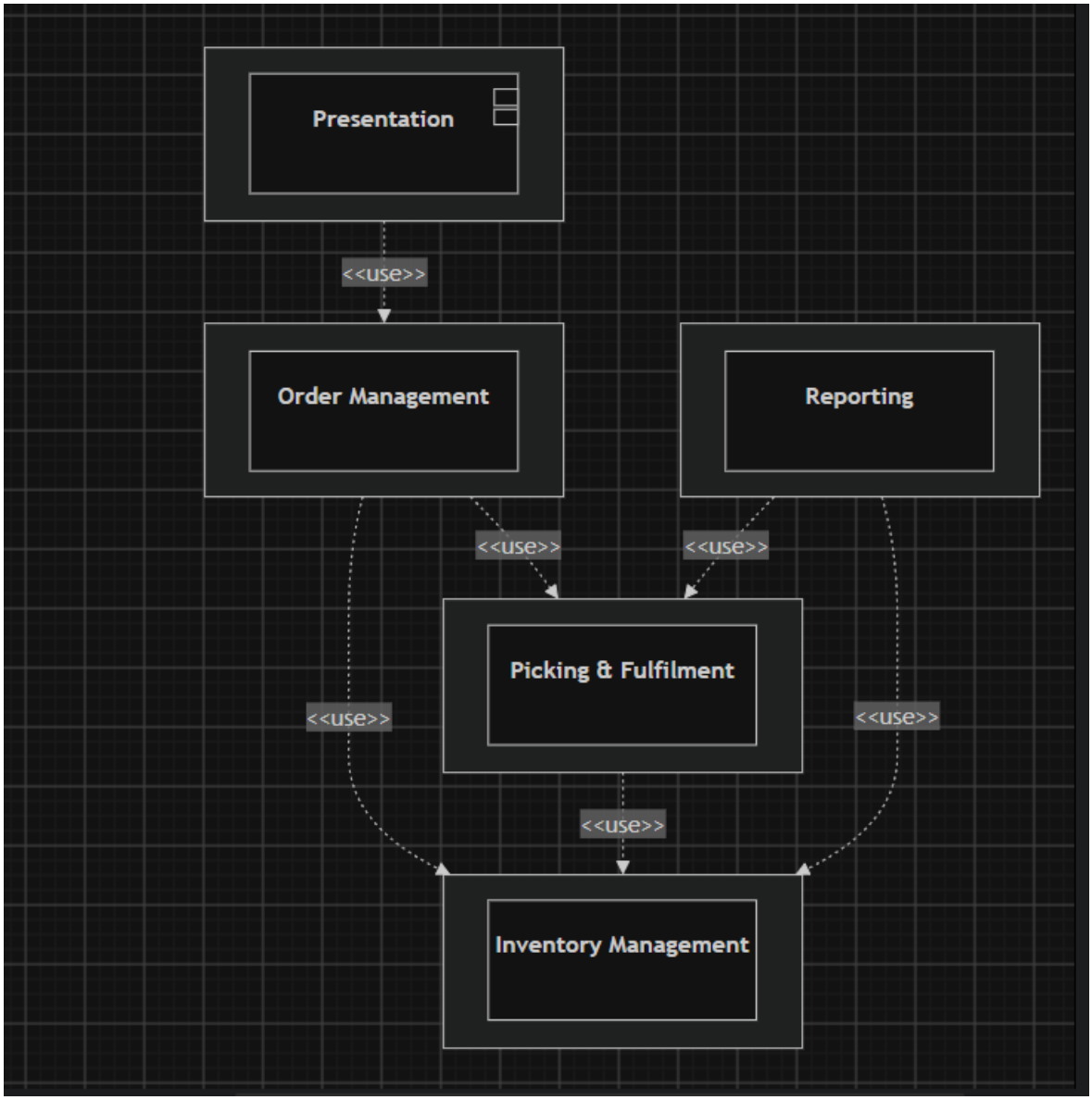

Figure 7. Package Diagram.

11. Component Diagram

The Warehouse Application Server exposes three internal services (Order, Inventory, Picking) as components with defined interfaces. The Handheld Scanner App and Reporting Dashboard depend on these interfaces rather than on the server's internal implementation.

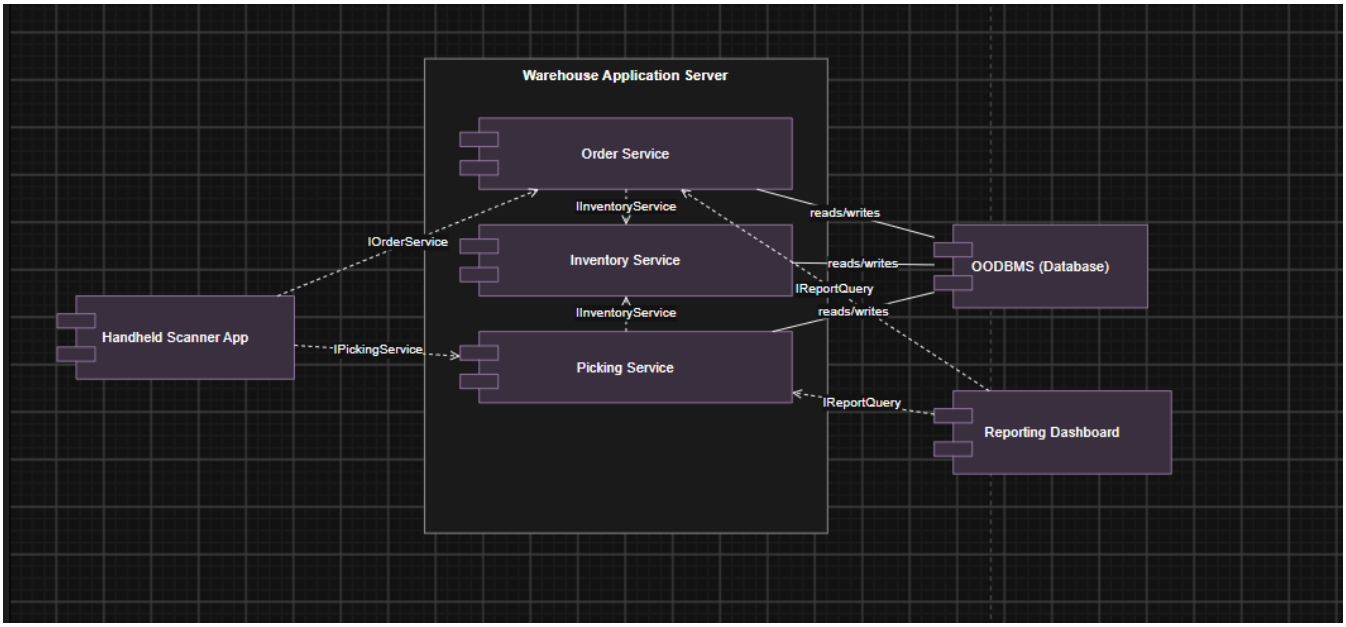

Figure 8. Component Diagram.

12. Deployment Diagram

Shows the physical/runtime nodes: a Handheld Scanner device communicating over HTTPS/REST with the Warehouse Application Server, which talks to the Database Server over JDBC/ODBC, and a browser-based Reporting Dashboard also communicating over HTTPS/REST.

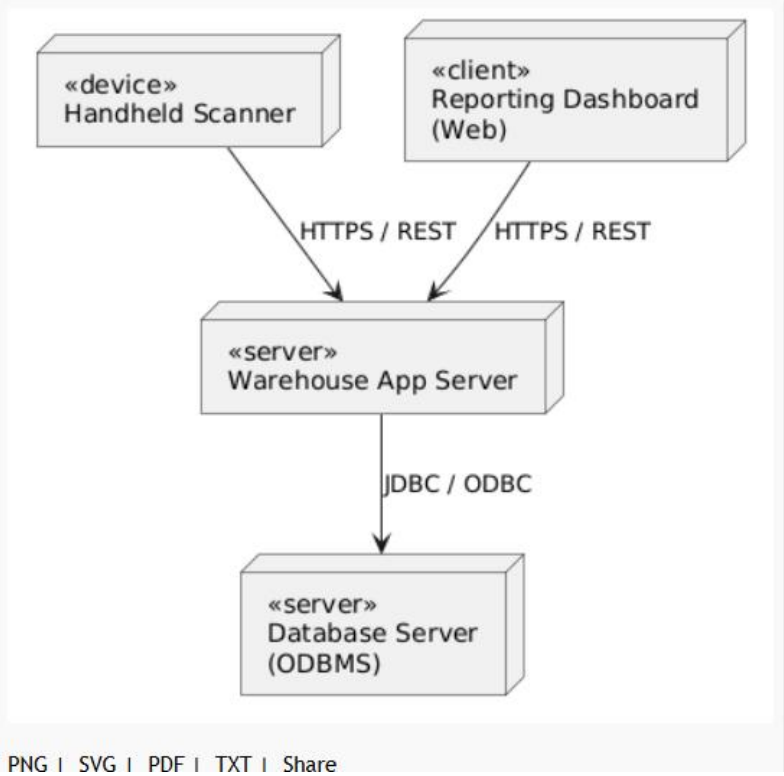

Figure 9. Deployment Diagram.

13. Object Responsibility Analysis

CRC-style analysis assigns each class a small, coherent set of responsibilities and identifies who it must collaborate with to fulfil them — this is what keeps every class in Section 5 focused on a single concern.

Class	Responsibilities	Collaborators
Order	Own its lifecycle status; validate that it can be confirmed; own its line items.	OrderLineItem, Customer, PickingSession
Item	Track its own stock level; decide whether it needs reordering.	StorageBin, OrderLineItem
StorageBin	Track its own occupancy; decide whether it can accept more stock.	Item
PickingSession	Track picking progress; decide when to move to Exception.	Picker, Order
OrderLineItem	Compute its own subtotal.	Item
Picker	Know which session it is currently assigned to.	PickingSession
Customer	Initiate an order on the customer's behalf.	Order

14. Design Axioms, Principles and Design Pattern Justification

Design Axioms

- **Independence Axiom —** each class satisfies one functional requirement without disturbing the others. Item's stock-management responsibility (FR1) is completely independent of Order's lifecycle-management responsibility (FR5): changing how reorder thresholds are checked never requires touching Order, and vice versa.
- **Information Axiom —** among designs that satisfy the independence axiom, the one with the least information content (complexity) is preferred. This is why StorageBin's capacity-checking logic is kept entirely inside StorageBin.checkAvailability() rather than duplicated inside Order or OrderLineItem — the simplest design keeps that knowledge in exactly one place.

Cohesion and Coupling

Every class in Section 5 exhibits high functional cohesion: PickingSession, for instance, only holds picking-related state and behaviour, never order-creation or inventory logic. Coupling between subsystems is kept low and explicit through the Package Diagram's <<use>> dependencies (Section 10) rather than classes reaching directly across package boundaries — Reporting depends on Order Management and Picking & Fulfilment only through their public operations, never their internal attributes.

Encapsulation and Abstraction

All attributes in the Class Diagram are private, changed only through the class's own methods (e.g. Item.updateStock()), and callers interact with abstract operations (Order.confirmOrder()) without needing to know how the underlying validation is implemented.

Design Pattern Justification

- **State Pattern — Order lifecycle:** rather than representing Order.status as a plain string checked with conditional logic scattered across the codebase, the State pattern is recommended: each lifecycle stage (Placed, Allocated, Picking, Packed, Dispatched, Delivered, Cancelled, Exception) becomes its own state class implementing a common OrderState interface, with each state class knowing only its own valid transitions. This maps directly onto the State Transition Diagram (Section 9) and keeps Order itself free of a large switch/if-else block that would otherwise grow every time a new state or transition rule is added.
- **Factory Method — Order subtype creation:** since the concrete class to instantiate (StandardOrder or ExpressOrder) depends on the customer's chosen priority at the moment Place Order (UC-4) executes, an OrderFactory with a createOrder(priority) method is recommended so that OrderController never contains an if/else on the priority string itself — adding a future third priority tier would mean extending the factory, not modifying every place an order is created.
- **Observer Pattern — order status notifications:** Track Order Status (UC-6) and the Reporting package both need to know whenever Order.status changes. Rather than having Order call Reporting and the UI directly (which would tightly couple Order to both), Order is recommended to notify a list of registered Observer objects on every state transition, letting the

Reporting package and any dashboard subscribe without Order needing to know they exist — directly supporting the low-coupling goal discussed above.

15. Assumptions and Constraints

Assumptions

- A picking session is created against one Order as a whole, not against individual line items — one picker completes all items on an order's pick list.
- A FulfilmentReport is a derived/computed view over Order and PickingSession data, not a persisted entity in its own right.
- Item-to-bin storage is simplified to one bin holding one or more units of a single item type at a time.

Constraints (from the Problem Statement)

- Item IDs, bin IDs, order IDs, and pick-list IDs must be unique — enforced by Item, StorageBin, Order, and PickingSession's own creation logic rejecting duplicates.
- A bin cannot be allocated beyond its available capacity — enforced by StorageBin.checkAvailability() before StorageBin.allocate() proceeds.
- An order cannot be confirmed if any line item exceeds available stock — enforced by the Check Stock Availability included use case (UC-4) calling Item.checkReorderNeeded()-adjacent stock validation before Order.confirmOrder().
- A picker cannot be assigned two active pick-lists simultaneously — enforced by Picker.getActiveSession() being checked before a new PickingSession is assigned.
- The design must scale to at least 200 items, 50 concurrent orders, and 10 storage zones without a fundamental redesign — addressed in the scalability discussion below.

16. Requirement-to-Use Case/Class Traceability

Requirement	Use Case(s)	Class(es)
FR1 — Item and Inventory Management	UC-1 Register Item, UC-2 Receive Stock	Item
FR2 — Storage Zone / Bin Management	UC-3 Allocate Storage Bin	StorageBin, Item
FR3 — Order Management	UC-4 Place Order	Order, StandardOrder, ExpressOrder, OrderLineItem, Customer
FR4 — Picking and Fulfilment Session	UC-5 Pick Order	PickingSession, Picker
FR5 — Order Status Tracking	UC-6 Track Order Status	Order (state chart, Section 9)
FR6 — Reporting	UC-7 Generate Fulfilment Report	Order, PickingSession, Item (aggregated in the Reporting package)

Every functional requirement traces to at least one use case and at least one class, and every class in Section 5 is exercised by at least one use case above, so no orphan classes or untraceable requirements exist in this design.

17. Implementation / Prototype and Results

The class skeletons below translate the Class Diagram (Section 6) directly into a Java-style prototype, intended as the starting point for the full implementation to be maintained in the GitHub repository (Section 19). They show the constructor and the key methods whose logic is dictated by the constraints in Section 15, rather than a complete, tested implementation.

```
public class Item {
    private String itemId;
    private int currentStock;
    private int reorderThreshold;

    public boolean checkReorderNeeded() {
        return currentStock < reorderThreshold;
    }

    public void updateStock(int delta) {
        this.currentStock += delta;
    }
}

public class StorageBin {
    private int maxCapacity;
    private int currentOccupancy;

    public boolean checkAvailability(int qty) {
        return (currentOccupancy + qty) <= maxCapacity;
    }

    public boolean allocate(int qty) {
        if (!checkAvailability(qty)) return false;
        currentOccupancy += qty;
        return true;
    }
}

public class Order {
    private String status = "Placed";

    public boolean confirmOrder(List<OrderLineItem> lines) {
        for (OrderLineItem li : lines) {
            if (li.getItem().getCurrentStock() < li.getQuantity()) {
                return false; // reject: exceeds available stock
            }
```

```
        }
        this.status = "Allocated";
        return true;
    }
}
```

Results and Testing Plan

Once implemented, the prototype should be exercised against the constraints in Section 15 with unit tests covering: duplicate item/bin/order/pick-list ID rejection, bin over-allocation rejection, order confirmation against insufficient stock, and double pick-list assignment rejection. Actual test results, coverage reports, and any UI screenshots are to be captured and committed to the GitHub repository rather than reproduced here, since they depend on code not yet written at the time of this design report.

18. Reflection on Design Decisions and Learning Outcomes

Design Decisions

The most consequential decision was keeping every business rule encapsulated inside the class that owns the data it protects (e.g. bin-capacity checking lives only in StorageBin), rather than validating rules externally in a controller — this is what let the traceability matrix in Section 16 map so cleanly, since each requirement's enforcement logic has exactly one home.

SDG Relevance

The design connects to SDG 9 (Industry, Innovation and Infrastructure) by digitising a manual, error-prone warehouse process into a scalable software system, and to SDG 12 (Responsible Consumption and Production) through the reorder-threshold and zone-utilisation reporting mechanisms, which directly support reducing overstocking and wastage.

Challenges and Learning Outcomes

Deciding how much of the design to express through inheritance (Order/StandardOrder/ExpressOrder) versus through composition or a design pattern (the State and Factory patterns discussed in Section 14) was the most challenging modelling decision, since both are valid ways to express the same requirement — this exercise showed that the choice should be driven by which axis of variation (order priority vs. lifecycle stage) is more likely to grow over time, not by habit.

19. GitHub Upload

The repository accompanying this report should contain: this design report, all nine UML diagram source files (editable, e.g. draw.io/.drawio or the CASE tool's native project file), the prototype class skeletons from Section 17 developed into a working implementation, and any test results and screenshots gathered once the prototype is exercised. A short README should point a reader to each of these in one place, mirroring the deliverables checklist below.

Deliverable	Location in This Report
Problem Analysis and Requirement Summary	Section 1
Actor Identification	Section 2
Use Case Descriptions	Section 3
Use Case Diagram	Section 4 (Figure 1)
Object and Class Identification	Section 5
Class Diagram	Section 6 (Figure 2)
Sequence/Interaction Diagram(s)	Section 7 (Figures 3-4)
Activity Diagram	Section 8 (Figure 5)
State Transition Diagram	Section 9 (Figure 6)
Package Diagram	Section 10 (Figure 7)
Component Diagram	Section 11 (Figure 8)
Deployment Diagram	Section 12 (Figure 9)
Object Responsibility Analysis	Section 13
Design Axioms/Principles and Pattern Justification	Section 14
Assumptions and Constraints	Section 15
Requirement-to-Use Case/Class Traceability	Section 16
Implementation/Prototype and Results	Section 17
Reflection on Design Decisions and Learning Outcomes	Section 18
GitHub Upload	https://github.com/192521357simats-cpu/SWISS-OOAD